

WEB APPLICATION DEVELOPMENT

LAB 8 - HOMEWORK

EXERCISE 5: SEARCH & FILTER ENDPOINTS

5.1: Search Customers

Update CustomerRepository:

```
@Query("SELECT c FROM Customer c WHERE " +
    "LOWER(c.fullName) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
    "LOWER(c.email) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
    "LOWER(c.customerCode) LIKE LOWER(CONCAT('%', :keyword, '%'))")
List<Customer> searchCustomers(@Param("keyword") String keyword);
```

Update Service:

```
List<CustomerResponseDTO> searchCustomers(String keyword);

@Override
public List<CustomerResponseDTO> searchCustomers(String keyword) {
    return customerRepository.searchCustomers(keyword)
        .stream()
        .map(this::convertToResponseDTO)
        .collect(Collectors.toList());
}
```

Add to Controller:

```
// GET search customers
@GetMapping("/search")
public ResponseEntity<List<CustomerResponseDTO>> searchCustomers(@RequestParam String keyword) {
    List<CustomerResponseDTO> customers = customerService.searchCustomers(keyword);
    return ResponseEntity.ok(customers);
}
```

Test: GET /api/customers/search?keyword=john

The screenshot displays the Thunder Client interface. The top panel shows a list of requests, with the selected request being a GET to `localhost:8080/api/customers/search?keyword=john`. The middle panel shows the request details, including the query parameters and the response body. The response is a JSON array of two customer objects. The bottom panel shows the terminal output, which displays the SQL query used for the search.

Request Details:

- Method: GET
- URL: `http://localhost:8080/api/customers/search?keyword=john`
- Status: 200 OK
- Size: 513 Bytes
- Time: 334 ms

Response Body:

```
[{"id": 1, "customerCode": "C001", "fullName": "John Doe", "email": "john.doe@example.com", "phone": "+1-555-0101", "address": "123 Main St, New York, NY 10001", "status": "ACTIVE", "createdAt": "2025-12-02T15:44:03"}, {"id": 3, "customerCode": "C003", "fullName": "Bob Johnson", "email": "bob.johnson@example.com", "phone": "+1-555-0103", "address": "789 Pine Rd, Chicago, IL 60601", "status": "ACTIVE", "createdAt": "2025-12-02T15:44:03"}]
```

SQL Query:

```
SELECT c1_0.fullName, c1_0.phone, c1_0.status, c1_0.updated_at
FROM customers c1_0
WHERE
    lower(c1_0.fullName) like lower(concat('%', ?, '%')) escape ''
    or lower(c1_0.email) like lower(concat('%', ?, '%')) escape ''
    or lower(c1_0.customer_code) like lower(concat('%', ?, '%')) escape ''
```

Explanation of the code flow of searching customers with keyword:

1. Browser / Frontend → sends HTTP GET request with keyword such as GET /api/customers/search?keyword=john.

2. CustomerRestController receives request (@GetMapping("/search")). Spring Boot:
 - Detects the route /api/customers/search
 - Reads query parameter → keyword = john
 - Calls the controller method
3. Controller calls service.searchCustomers(keyword).
 - Controller again does not perform any logic.
 - Delegates the processing to the Service Layer.
4. Service calls repository.searchCustomers(keyword)
5. JPA generates SQL query behind the scenes
 - This allows case-insensitive searching.
 - Matches anywhere in the text using %keyword%.
6. Database returns matching Customer entities. These are mapped into: List<Customer> and returned to the service.
7. Service converts Entity → ResponseDTO (id, customerCode, fullName, email, phone, address, status, createdAt).
8. Controller returns ResponseEntity with the list
9. Spring Boot converts DTO list → JSON
10. Frontend (Thunder Client) receives the JSON and displays filtered results

5.2: Filter by Status

Add to Repository:

```
List<Customer> findByStatus(Customer.CustomerStatus status);
```

Add to Controller:

```
// GET customers by status
@GetMapping("/status/{status}")
public ResponseEntity<List<CustomerResponseDTO>> getCustomersByStatus(@PathVariable String status) {
    List<CustomerResponseDTO> customers = customerService.getCustomersByStatus(status);
    return ResponseEntity.ok(customers);
}
```

Add to Service:

```
List<CustomerResponseDTO> getCustomersByStatus(String status);

@Override
public List<CustomerResponseDTO> getCustomersByStatus(String status) {
    Customer.CustomerStatus enumStatus;

    try {
        enumStatus = Customer.CustomerStatus.valueOf(status.toUpperCase());
    } catch (IllegalArgumentException ex) {
        throw new ResourceNotFoundException("Invalid status: " + status);
    }

    return customerRepository.findByStatus(enumStatus)
        .stream()
        .map(this::convertToResponseDTO)
        .collect(Collectors.toList());
}
```

Test:

- GET /api/customers/status/ACTIVE

Thunder Client interface showing a GET request to `localhost:8080/api/customers/status/ACTIVE`. The response is a JSON array of 3 active customers. The terminal shows a SQL query filtering by status.

```

GET localhost:8080/api/customers/status/ACTIVE
Status: 200 OK Size: 1.01 KB Time: 29 ms
Response:
[
  {
    "id": 1,
    "customerCode": "C001",
    "fullName": "John Doe",
    "email": "john.doe@example.com",
    "phone": "+1-555-0101",
    "address": "123 Main St, New York, NY 10001",
    "status": "ACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  },
  {
    "id": 2,
    "customerCode": "C002",
    "fullName": "Jane Smith",
    "email": "jane.smith@example.com",
    "phone": "+1-555-0102",
    "address": "456 Oak Ave, Los Angeles, CA 90001",
    "status": "ACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  },
  {
    "id": 3,
    "customerCode": "C003",
    "fullName": "Bob Johnson",
    "email": "bob.johnson@example.com",
    "phone": "+1-555-0103",
    "address": "789 Pine St, Chicago, IL 60601",
    "status": "ACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  }
]

```

```

SELECT c1_0.customer_code,
c1_0.email,
c1_0.full_name,
c1_0.phone,
c1_0.status,
c1_0.updated_at
FROM customers c1_0
WHERE c1_0.status=?

```

– GET /api/customers/status/INACTIVE:

Thunder Client interface showing a GET request to `localhost:8080/api/customers/status/INACTIVE`. The response is a JSON array of 1 inactive customer. The terminal shows a SQL query filtering by status.

```

GET localhost:8080/api/customers/status/INACTIVE
Status: 200 OK Size: 261 Bytes Time: 12 ms
Response:
[
  {
    "id": 4,
    "customerCode": "C004",
    "fullName": "Alice Brown",
    "email": "alice.brown@example.com",
    "phone": "+1-555-0104",
    "address": "321 Elm St, Houston, TX 77001",
    "status": "INACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  }
]

```

```

SELECT c1_0.customer_code,
c1_0.email,
c1_0.full_name,
c1_0.phone,
c1_0.status,
c1_0.updated_at
FROM customers c1_0
WHERE c1_0.status=?

```

Explanation of the code flow of get customer status:

1. Browser → GET /api/customers/status/ACTIVE: The frontend sends a request to filter customers whose status is **ACTIVE**.
2. **CustomerRestController** receives request (`@GetMapping("/status/{status}")`). Spring Boot extracts: `status = "ACTIVE"`
3. Controller calls `customerService.getCustomersByStatus(status)`: The controller passes the string "ACTIVE" to the service layer.
4. Service converts String → Enum:
`enumStatus = CustomerStatus.valueOf(status.toUpperCase());`

- If status is not valid → throws exception → 404 Not Found.
5. Service calls `customerRepository.findByStatus(enumStatus)`: Now Spring Data JPA receives the correct type: `Customer.CustomerStatus`.
 6. JPA automatically builds SQL WHERE status = 'ACTIVE'
 7. Repository returns `List<Customer>`. It returns all customers whose status matches the enum.
 8. Service converts entities → `List<CustomerResponseDTO>`
 9. Controller returns `ResponseEntity.ok(customerList)`
 - HTTP 200
 - Body = JSON array of customers
 10. Spring Boot converts DTO list → JSON
 11. JSON response sent to browser/client (Thunder Client).

5.3: Advanced Search with Multiple Criteria

Add to Repository:

```
@Query("SELECT c FROM Customer c " +
    "WHERE (:name IS NULL OR LOWER(c.fullName) LIKE LOWER(CONCAT('%', :name, '%'))) " +
    "AND (:email IS NULL OR LOWER(c.email) LIKE LOWER(CONCAT('%', :email, '%'))) " +
    "AND (:status IS NULL OR c.status = :status)")
List<Customer> advancedSearch(
    @Param("name") String name,
    @Param("email") String email,
    @Param("status") Customer.CustomerStatus status
);
```

Add to Service:

```
List<CustomerResponseDTO> advancedSearch(String name, String email, String status);

@Override
public List<CustomerResponseDTO> advancedSearch(String name, String email, String status) {
    Customer.CustomerStatus enumStatus = null;
    // Convert status string → enum
    if (status != null && !status.isBlank()) {
        try {
            enumStatus = Customer.CustomerStatus.valueOf(status.toUpperCase());
        } catch (IllegalArgumentException ex) {
            throw new ResourceNotFoundException("Invalid status: " + status);
        }
    }

    List<Customer> customers = customerRepository.advancedSearch(
        (name == null || name.isBlank()) ? null : name,
        (email == null || email.isBlank()) ? null : email,
        enumStatus
    );

    return customers.stream()
        .map(this::convertToResponseDTO)
        .collect(Collectors.toList());
}
```

Add to Controller:

```
// GET customers by advanced search
@GetMapping("/advanced-search")
public ResponseEntity<List<CustomerResponseDTO>> advancedSearch(
    @RequestParam(required = false) String name,
    @RequestParam(required = false) String email,
    @RequestParam(required = false) String status) {

    List<CustomerResponseDTO> customers =
        customerService.advancedSearch(name, email, status);

    return ResponseEntity.ok(customers);
}
```

Test:

- GET /api/customers/advanced-search?email=gmail&status=ACTIVE

The screenshot shows the Thunder Client interface. The 'Activity' panel on the left lists several requests. The selected request is a GET to `localhost:8080/api/customers/advanced-search?email=john&status=ACTIVE` from 4 minutes ago. The 'Query' panel shows parameters: `email=john` and `status=ACTIVE`. The 'Response' panel shows a 200 OK status with 513 bytes and 146 ms time. The response body is a JSON array of two customer objects:

```
[
  {
    "id": 1,
    "customerCode": "C001",
    "fullName": "John Doe",
    "email": "john.doe@example.com",
    "phone": "+1-555-0101",
    "address": "123 Main St, New York, NY 10001",
    "status": "ACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  },
  {
    "id": 3,
    "customerCode": "C003",
    "fullName": "Bob Johnson",
    "email": "bob.johnson@example.com",
    "phone": "+1-555-0103",
    "address": "789 Pine Rd, Chicago, IL 60601",
    "status": "ACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  }
]
```

The 'Terminal' panel at the bottom shows the SQL query executed:

```
select
  c1_0.id,
  c1_0.address,
  c1_0.created_at,
  c1_0.customer_code,
  c1_0.email,
  c1_0.full_name,
  c1_0.phone,
  c1_0.status,
  c1_0.updated_at
from
  customers c1_0
```

- localhost:8080/api/customers/advanced-search?name=alice&status=INACTIVE

The screenshot shows the Thunder Client interface. The 'Activity' panel on the left lists several requests. The selected request is a GET to `localhost:8080/api/customers/advanced-search?name=alice&status=INACTIVE` from 'just now'. The 'Query' panel shows parameters: `name=alice` and `status=INACTIVE`. The 'Response' panel shows a 200 OK status with 261 bytes and 21 ms time. The response body is a JSON array with one customer object:

```
[
  {
    "id": 4,
    "customerCode": "C004",
    "fullName": "Alice Brown",
    "email": "alice.brown@example.com",
    "phone": "+1-555-0104",
    "address": "321 Elm St, Houston, TX 77001",
    "status": "INACTIVE",
    "createdAt": "2025-12-02T15:44:03"
  }
]
```

The 'Terminal' panel at the bottom shows the SQL query executed:

```
select
  c1_0.id,
  c1_0.address,
  c1_0.created_at,
  c1_0.customer_code,
  c1_0.email,
  c1_0.full_name,
  c1_0.phone,
  c1_0.status,
  c1_0.updated_at
from
  customers c1_0
```

Explanation of the code flow of the advanced search (with name, email, and status)

1. Browser → GET /api/customers/advanced-search?name=john&status=ACTIVE
 - The frontend sends multiple search filters.
 - All parameters are optional, so any combination is valid
2. CustomerRestController receives request (@GetMapping("/advanced-search"))
 - Spring Boot extracts name = john, email = null, status = "ACTIVE"

3. Controller calls `customerService.advancedSearch(name, email, status)`. Controller does not apply logic; it forwards parameters to the service.
4. Service processes each parameter (check null / empty). Service determines which filters are active. For example:
 - name → filter enabled
 - email → filter disabled
 - status → convert string → enum
5. Service builds dynamic query. The service calls a custom repository method like: `customerRepository.advancedSearch(name, email, enumStatus);`.
6. **JPA generates SQL using only provided filters**
 - Example SQL: `SELECT * FROM customers WHERE LOWER(full_name) LIKE '%john%' AND status = 'ACTIVE';`.
 - If email was provided, it adds: `AND LOWER(email) LIKE '%gmail%'`
7. Repository returns `List<Customer>` that match all criteria. Only customers matching all provided filters are returned.
8. Service converts each `Customer` → `CustomerResponseDTO` using `.stream().map(this::convertToResponseDTO)`
9. Controller returns `ResponseEntity.ok(listOfDTOs)`
 - Status: **200 OK**
 - Body: JSON list of matching customers.
10. Spring Boot converts DTO list → JSON automatically
11. JSON response sent back to browser

EXERCISE 6: PAGINATION & SORTING

6.1: Add Pagination

Update Service to use Pageable:

```
Page<CustomerResponseDTO> getAllCustomers(int page, int size);

@Override
public Page<CustomerResponseDTO> getAllCustomers(int page, int size) {
    Pageable pageable = PageRequest.of(page, size);

    Page<Customer> customerPage = customerRepository.findAll(pageable);

    // Convert each Customer → CustomerResponseDTO
    return customerPage.map(this::convertToResponseDTO);
}
```

Update Controller:

```
@GetMapping
public ResponseEntity<Map<String, Object>> getAllCustomers(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size) {

    Page<CustomerResponseDTO> customerPage = customerService.getAllCustomers(page, size);

    Map<String, Object> response = new HashMap<>();
    response.put(key: "customers", customerPage.getContent());
    response.put(key: "currentPage", customerPage.getNumber());
    response.put(key: "totalItems", customerPage.getTotalElements());
    response.put(key: "totalPages", customerPage.getTotalPages());

    return ResponseEntity.ok(response);
}
```


Test:

GET /api/customers?page=0&size=5

The screenshot shows the VS Code interface with a REST client request and response. The request is a GET request to `localhost:8080/api/customers?page=0&size=5`. The response is a 200 OK status with a size of 1.44 KB and a time of 48 ms. The response body is a JSON array of customer objects.

```
GET localhost:8080/api/customers?page=0&size=5
```

Query Parameters:

- page: 0
- size: 5

Response:

```
{
  "totalItems": 5,
  "totalPages": 1,
  "customers": [
    {
      "id": 1,
      "customerCode": "C001",
      "fullName": "John Doe",
      "email": "john.doe@example.com",
      "phone": "+1-555-0101",
      "address": "123 Main St, New York, NY 10001",
      "status": "ACTIVE",
      "createdAt": "2025-12-02T15:44:03"
    },
    {
      "id": 2,
      "customerCode": "C002",
      "fullName": "Jane Smith",
      "email": "jane.smith@example.com",
      "phone": "+1-555-0102",
      "address": "456 Oak Ave, Los Angeles, CA 90001",
      "status": "ACTIVE",
      "createdAt": "2025-12-02T15:44:03"
    }
  ]
}
```

GET /api/customers?page=1&size=10

The screenshot shows the VS Code interface with a REST client request and response. The request is a GET request to `localhost:8080/api/customers?page=1&size=10`. The response is a 200 OK status with a size of 89 Bytes and a time of 28 ms. The response body is a JSON array of customer objects.

```
GET localhost:8080/api/customers?page=1&size=10
```

Query Parameters:

- page: 1
- size: 10

Response:

```
{
  "totalItems": 5,
  "totalPages": 1,
  "customers": [],
  "currentPage": 1
}
```

localhost:8080/api/customers?page=1&size=2

The screenshot shows the VS Code interface with a REST client request and response. The request is a GET request to `localhost:8080/api/customers?page=1&size=2`. The response is a 200 OK status with a size of 641 Bytes and a time of 22 ms. The response body is a JSON array of customer objects.

```
GET localhost:8080/api/customers?page=1&size=2
```

Query Parameters:

- page: 1
- size: 2

Response:

```
{
  "totalItems": 5,
  "totalPages": 3,
  "customers": [
    {
      "id": 3,
      "customerCode": "C003",
      "fullName": "Bob Johnson",
      "email": "bob.johnson@example.com",
      "phone": "+1-555-0103",
      "address": "789 Pine Rd, Chicago, IL 60601",
      "status": "ACTIVE",
      "createdAt": "2025-12-02T15:44:03"
    },
    {
      "id": 4,
      "customerCode": "C004",
      "fullName": "Alice Brown",
      "email": "alice.brown@example.com",
      "phone": "+1-555-0104",
      "address": "321 Elm St, Houston, TX 77001",
      "status": "INACTIVE",
      "createdAt": "2025-12-02T15:44:03"
    }
  ]
}
```

Terminal Output:

```
c1_0.status,
c1_0.updated_at
from
customers c1_0
limit
?, ?
Hibernate:
select
count(c1_0.id)
from
customers c1_0
```

Explanation of the code flow of the pagination:

1. Browser → GET /api/customers?page=0&size=5: The user enters a URL or Postman sends:
<http://localhost:8080/api/customers?page=0&size=5>
 - page=0 → first page (Spring Boot pages start at index 0)
 - size=5 → 5 customers per page
2. Controller reads page + size parameters
 - Spring automatically extracts page and size from the URL.
 - If user doesn't supply values → default values are used.
3. Controller calls `customerService.getAllCustomers(page, size)`
 - The controller passes the extracted parameters to the service
4. Service creates Pageable
 - In the service: `Pageable pageable = PageRequest.of(page, size);`
5. Repository runs paginated SQL query
 - Spring Data JPA converts this into SQL like:
`SELECT * FROM customers LIMIT 5 OFFSET 0;`
6. JPA returns `Page<Customer>`. This contains the list of customers in the current page, page index, total rows in database, total pages, page size.
7. Service maps entity → DTO
 - The service converts all customers into DTOs:
`customerPage.map(this::convertToResponseDTO);`
 - This ensures the controller returns DTO, not Entity.
8. Controller builds pagination metadata
9. Returns JSON containing:
 - customers
 - currentPage
 - totalItems
 - totalPages
10. Browser receives paginated data: Postman or the frontend receives:
 - Only 5 customers
 - And pagination metadata (so UI can show page numbers, next/previous)

6.2: Add Sorting

Update Controller:

```
@GetMapping
public ResponseEntity<Map<String, Object>> getAllCustomers(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "id") String sortBy,
    @RequestParam(defaultValue = "asc") String sortDir) {

    Page<CustomerResponseDTO> customerPage =
        customerService.getAllCustomers(page, size, sortBy, sortDir);

    Map<String, Object> response = new HashMap<>();
    response.put(key: "customers", customerPage.getContent());
    response.put(key: "currentPage", customerPage.getNumber());
    response.put(key: "totalItems", customerPage.getTotalElements());
    response.put(key: "totalPages", customerPage.getTotalPages());
    response.put(key: "sortBy", sortBy);
    response.put(key: "sortDir", sortDir);

    return ResponseEntity.ok(response);
}
```

Add to Service:


```

Page<CustomerResponseDTO> getAllCustomers(int page, int size, String sortBy, String sortDir);

@Override
public Page<CustomerResponseDTO> getAllCustomers(int page, int size, String sortBy, String sortDir) {

    Sort sort = sortDir.equalsIgnoreCase(anotherString: "asc")
        ? Sort.by(sortBy).ascending()
        : Sort.by(sortBy).descending();

    Pageable pageable = PageRequest.of(page, size, sort);

    Page<Customer> customerPage = customerRepository.findAll(pageable);

    return customerPage.map(this::convertToResponseDTO);
}

```

Test:

GET /api/customers?sortBy=fullName&sortDir=asc

The screenshot shows an IDE with a REST client window. The request is a GET to `localhost:8080/api/customers?page=0&size=5&sortBy=fullName&sortDir=asc`. The response is a 200 OK with a JSON body containing a list of 2 customers. The first customer is Alice Brown and the second is Bob Johnson.

```

{
  "totalItems": 5,
  "totalPages": 1,
  "sortBy": "fullName",
  "customers": [
    {
      "id": 4,
      "customerCode": "C004",
      "fullName": "Alice Brown",
      "email": "alice.brown@example.com",
      "phone": "+1-555-0104",
      "address": "321 Elm St, Houston, TX 77001",
      "status": "INACTIVE",
      "createdAt": "2025-12-02T15:44:03"
    },
    {
      "id": 3,
      "customerCode": "C003",
      "fullName": "Bob Johnson",
      "email": "bob.johnson@example.com",
      "phone": "+1-555-0103",
      "address": "789 Pine Rd, Chicago, IL 60601",
      "status": "ACTIVE",
      "createdAt": "2025-12-02T15:44:03"
    }
  ]
}

```

GET /api/customers?sortBy=createdAt&sortDir=desc

The screenshot shows an IDE with a REST client window. The request is a GET to `localhost:8080/api/customers?sortBy=createdAt&sortDir=desc`. The response is a 200 OK with a JSON body containing a list of 2 customers. The first customer is David Miller and the second is John Doe.

```

{
  "totalItems": 6,
  "totalPages": 1,
  "sortBy": "createdAt",
  "customers": [
    {
      "id": 7,
      "customerCode": "C006",
      "fullName": "David Miller",
      "email": "david.miller@example.com",
      "phone": "+9847592378",
      "address": "999 Broadway, Seattle, WA 98101",
      "status": "ACTIVE",
      "createdAt": "2025-12-07T16:38:49"
    },
    {
      "id": 1,
      "customerCode": "C001",
      "fullName": "John Doe",
      "email": "john.doe@example.com",
      "phone": "+1-555-0101",
      "address": "123 Main St, New York, NY 10001",
      "status": "ACTIVE",
      "createdAt": "2025-12-02T15:44:03"
    }
  ]
}

```

Explanation of the code flow of sorting:

1. Browser → GET /api/customers?sortBy=fullName&sortDir=asc: The client includes two query parameters:
 - sortBy → the field to sort (example: fullName)
 - sortDir → direction (asc or desc).
2. Controller reads sorting parameters
 - Spring automatically extracts: sortBy=fullName, sortDir=asc
 - If user does not send these parameters: sortBy=null, sortDir=asc
3. Controller builds Sort object: This converts the request parameters into a Sort object understood by Spring Data JPA.
4. Controller calls customerService.getAllCustomers(sort): The controller passes the sort of configuration to the service.
5. Service calls Repository with sorting: Spring Data JPA automatically builds a SQL query including ORDER BY.
6. Repository executes SQL with ORDER BY
7. Service maps entities → DTO
8. Controller returns sorted list to browser
9. Browser receives sorted data

6.3: Combine Pagination and Sorting

Update the Controller:

```
@GetMapping
public ResponseEntity<Map<String, Object>> getAllCustomers(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "id") String sortBy,
    @RequestParam(defaultValue = "asc") String sortDir) {

    Page<CustomerResponseDTO> customerPage =
        customerService.getAllCustomers(page, size, sortBy, sortDir);

    Map<String, Object> response = new HashMap<>();
    response.put(key: "customers", customerPage.getContent());
    response.put(key: "currentPage", customerPage.getNumber());
    response.put(key: "totalItems", customerPage.getTotalElements());
    response.put(key: "totalPages", customerPage.getTotalPages());
    response.put(key: "sortBy", sortBy);
    response.put(key: "sortDir", sortDir);

    return ResponseEntity.ok(response);
}
```

Test:

GET /api/customers?page=0&size=5&sortBy=fullName&sortDir=asc

The screenshot shows an IDE with a REST client window. The request is a GET to localhost:8080/api/customers?page=0&size=5&sortBy=fullName&sortDir=asc. The response is a 200 OK status with a JSON body containing pagination information and a list of two customers.

Query Parameters	Value
page	0
size	5
sortBy	fullName
sortDir	asc

```
Response
1 {
2   "totalItems": 6,
3   "totalPages": 2,
4   "sortBy": "fullName",
5   "customers": [
6     {
7       "id": 4,
8       "customerCode": "C004",
9       "fullName": "Alice Brown",
10      "email": "alice.brown@example.com",
11      "phone": "+1-555-0104",
12      "address": "321 Elm St, Houston, TX 77001",
13      "status": "INACTIVE",
14      "createdAt": "2025-12-02T15:44:03"
15     },
16     {
17       "id": 3,
18       "customerCode": "C003",
19       "fullName": "Bob Johnson",
20       "email": "bob.johnson@example.com",
21       "phone": "+1-555-0103",
22       "address": "789 Pine Rd, Chicago, IL 60601",
23       "status": "ACTIVE",
24       "createdAt": "2025-12-02T15:44:03"
25     }
26   ]
27 }
```

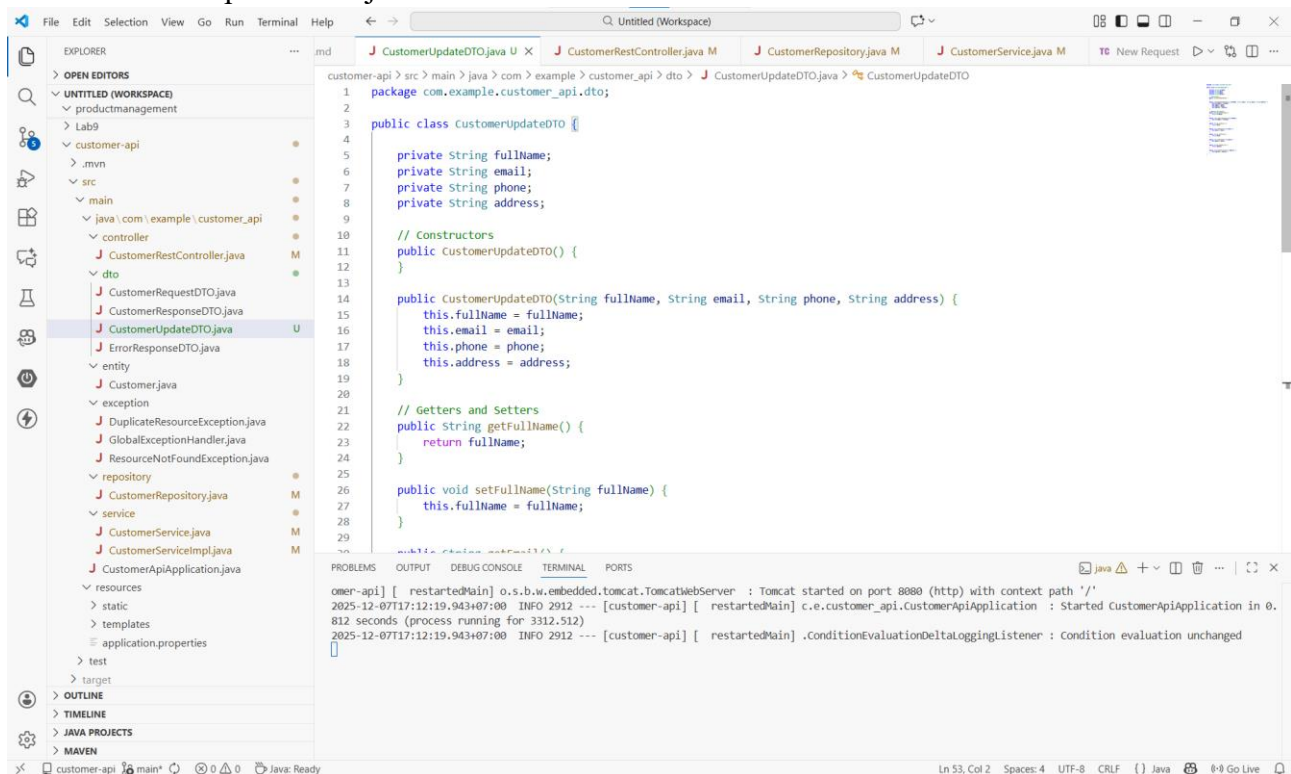
Explanation of the code flow of the combination the pagination and sorting

1. Browser → GET request with pagination + sorting parameters: User or frontend sends: `/api/customers?page=0&size=5&sortBy=fullName&sortDir=asc`. This contains four query parameters including page, size, sortBy, and sortDir.
2. Controller extracts page, size, sortBy, sortDir.
3. Controller constructs a Sort object
4. Controller builds a Pageable object combining both features. The Pageable object includes page number, page size, sorting rules.
5. Controller calls the service
6. Service sends Pageable to Repository. Spring Data JPA generates an optimized SQL query.
7. Repository executes SQL with LIMIT/OFFSET + ORDER BY
8. JPA returns a Page<Customer> object. Repository returns a Page object containing content (list of customers), number (current page), totalElements (all rows in database), totalPages (based on size), size.
9. Service maps Customer → CustomerResponseDTO
10. Controller prepares JSON response with metadata
11. Response returned to browser as JSON
12. Browser displays sorted + paginated results.

EXERCISE 7: PARTIAL UPDATE WITH PATCH

7.1: Create Update DTO

File: CustomerUpdateDTO.java



The screenshot shows an IDE with the following components:

- EXPLORER:** Displays the project structure. The 'dto' package under 'customer-api' is expanded, showing 'CustomerUpdateDTO.java' as the active file.
- EDITOR:** Contains the code for 'CustomerUpdateDTO.java'. The code defines a class with private fields for fullName, email, phone, and address. It includes a no-argument constructor, a constructor with all four fields, a getter for fullName, and a setter for fullName.
- PROBLEMS:** Empty.
- OUTPUT:** Shows the application startup logs, including 'Tomcat started on port 8080' and 'Started CustomerApiApplication in 0.812 seconds'.
- DEBUG CONSOLE:** Empty.
- TERMINAL:** Empty.
- PORTS:** Shows '8080' as an open port.

7.2: Implement PATCH Endpoint

Add to Controller:

```
@PatchMapping("/{id}")
public ResponseEntity<CustomerResponseDTO> partialUpdateCustomer(
    @PathVariable Long id,
    @RequestBody CustomerUpdateDTO updatedDTO) {

    CustomerResponseDTO updated = customerService.partialUpdateCustomer(id, updatedDTO);
    return ResponseEntity.ok(updated);
}
```

Add to Service:

`CustomerResponseDTO partialUpdateCustomer(Long id, CustomerUpdateDTO updateDTO);`

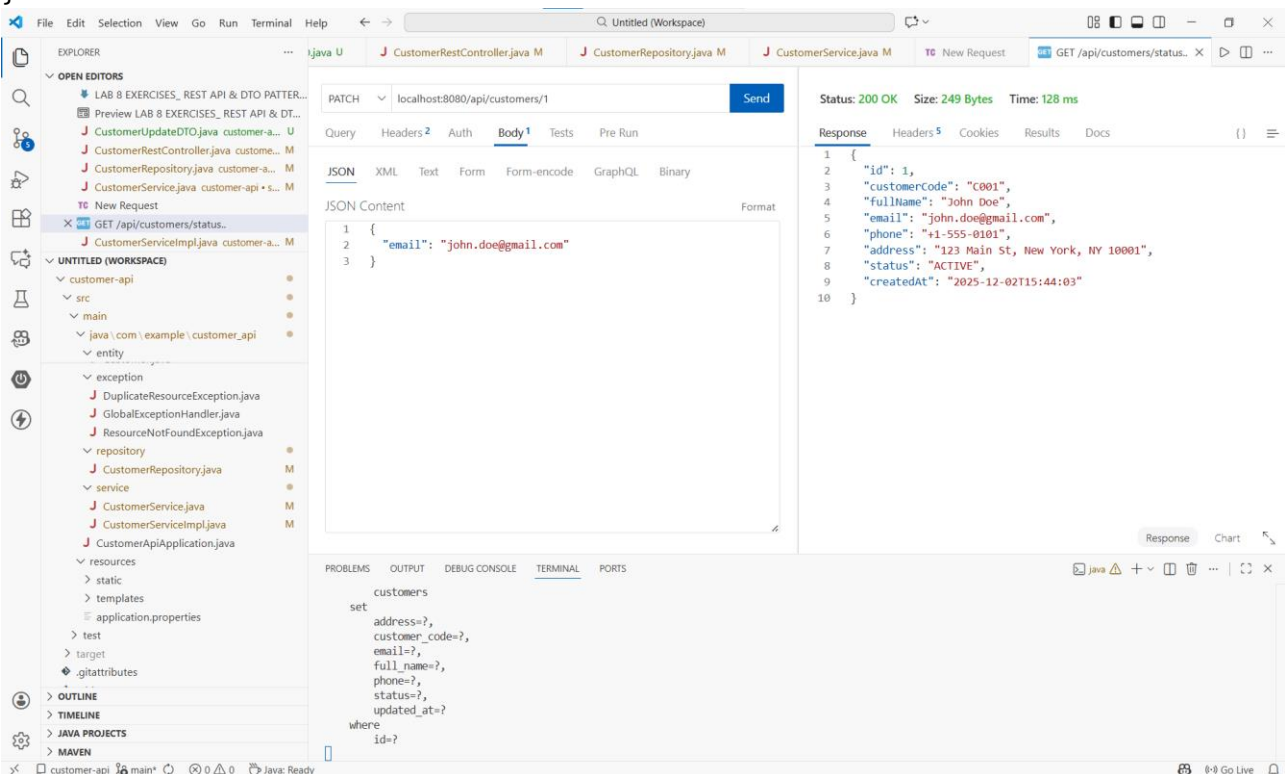
```
@Override
public CustomerResponseDTO partialUpdateCustomer(Long id, CustomerUpdateDTO updateDTO) {
    Customer customer = customerRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Customer not found with id: " + id));
    // Only update fields that are not null
    if (updateDTO.getFullName() != null) {
        customer.setFullName(updateDTO.getFullName());
    }
    if (updateDTO.getEmail() != null) {
        // Check duplicate email
        if (customerRepository.existsByEmail(updateDTO.getEmail()) &&
            !customer.getEmail().equals(updateDTO.getEmail())) {
            throw new DuplicateResourceException("Email already exists: " + updateDTO.getEmail());
        }
        customer.setEmail(updateDTO.getEmail());
    }
    if (updateDTO.getPhone() != null) {
        customer.setPhone(updateDTO.getPhone());
    }
    if (updateDTO.getAddress() != null) {
        customer.setAddress(updateDTO.getAddress());
    }
    // Save updated customer
    Customer saved = customerRepository.save(customer);
    return convertToResponseDTO(saved);
}
```

7.3: Test PATCH vs PUT

Test cases: localhost:8080/api/customers/1

JSON Content:

```
{
  "email": "john.doe@gmail.com"
}
```



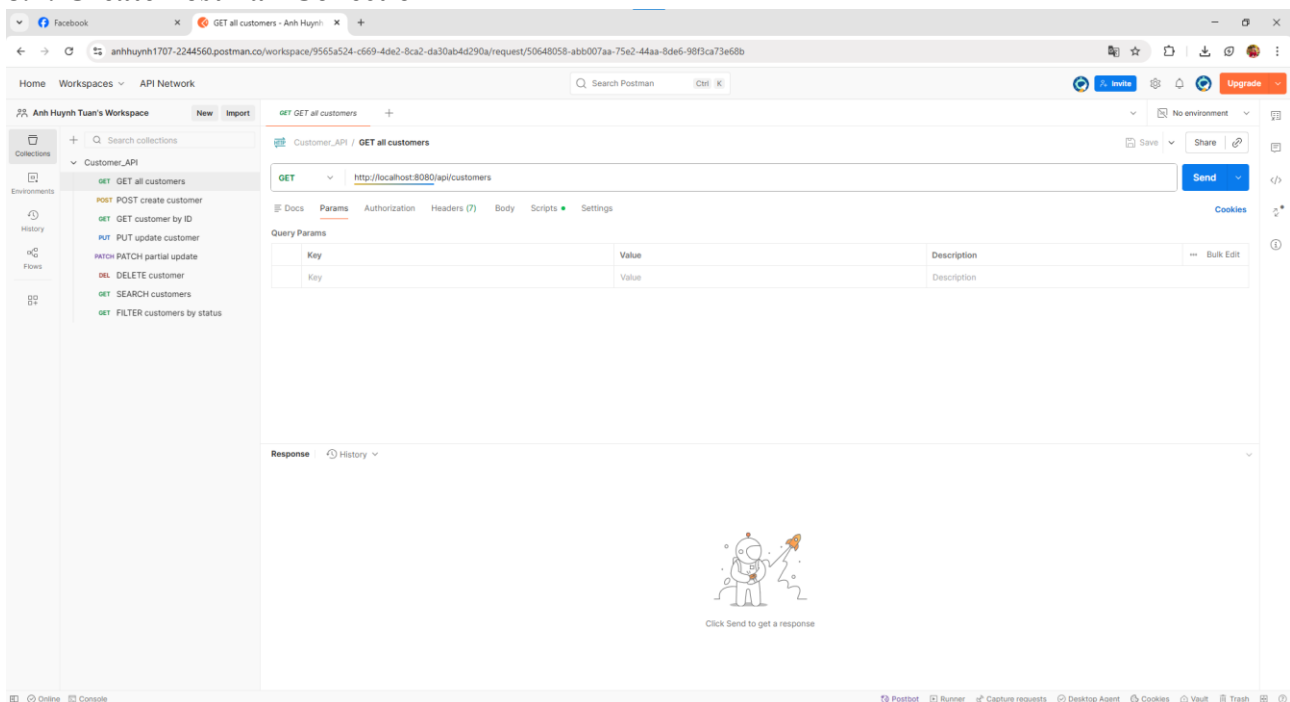
Explanation of the code flow of partial updating the customer with a patch:

1. Browser → Sends PATCH request with partial fields. The user or frontend sends

- HTTP method: PATCH
 - URL: /api/customers/5
 - Request body containing only the fields to update such as “email”: “john@gmail.com”.
2. Controller receives PATCH request. Spring extracts id, CustomerUpdateDTO.java file containing only the provided fields (others are null).
 3. Controller calls the Service method. The controller does not decide what to update, it only forwards the request to the service layer.
 4. Service retrieves existing customer from the database.
 - If ID exists → customer object retrieved successfully.
 - If ID does not exist → 404 Not Found is thrown.
 5. Service checks each field in CustomerUpdateDTO.java file
 - If field was included in → Update it
 - If field was not included in → Ignore it
 - Existing values remain unchanged.
 6. Service saves only the modified fields to the database.
 7. Service converts update entity to CustomerResponseDTO. DTO ensures that no internal fields are exposed and cleaned API response.
 8. Controller returns HTTP 200 OK with updated data
 9. Browser receives updated in few fields.

EXERCISE 8: API DOCUMENTATION

8.1: Create Postman Collection



8.2: Document API Responses

See on Github for preview (API_DOCUMENTATION.md)

8.3: Add Examples for Each Status Code

See on Github for preview (API_DOCUMENTATION.md)